

Estimativa estocástica de π com OpenMP

Matheus Marinho

3 de setembro de 2026

1. Objetivo e método

O objetivo foi estimar π por Monte Carlo, paralelizar o cálculo com OpenMP, observar uma condição de corrida e corrigi-la com `critical`. Também foram testadas as cláusulas `private`, `firstprivate`, `lastprivate`, `shared` e `default(none)`.

São gerados pontos uniformes no quadrado $[-1,1] \times [-1,1]$. Um ponto pertence ao círculo unitário quando $x^2 + y^2 \leq 1$. Como a área do círculo é π e a área do quadrado é 4, a estimativa é:

$$\hat{\pi} = 4 \cdot \text{pontos dentro do círculo} / \text{total de pontos}$$

Cada ponto é gerado a partir da semente e do índice da amostra. Assim, não existe um estado aleatório global sendo alterado pelas threads, e todas as versões usam os mesmos pontos. Mesmo quando o programa está correto, o resultado é apenas uma estimativa; em geral, o erro estatístico diminui quando o número de amostras aumenta.

2. Condição de corrida

A primeira versão paralela apenas acrescenta a diretiva solicitada:

```
#pragma omp parallel for
for (long long i = 0; i < amostras; i++) {
    /* geração e teste do ponto */
    if (x * x + y * y <= 1.0) {
        dentro++;
    }
}
```

`parallel` cria a equipe de threads e `for` distribui as iterações entre elas. A forma combinada `parallel for` executa essas duas ações em uma única diretiva.

O índice do laço e as variáveis declaradas dentro dele são privados, mas `dentro` é compartilhado. `dentro++` envolve leitura, soma e escrita. Duas threads podem ler o mesmo valor e gravar o mesmo sucessor, perdendo um incremento. Portanto, o resultado depende da ordem imprevisível dos acessos: isso é uma condição de corrida. Obter o resultado correto em uma execução, por exemplo com uma thread, não prova que a versão paralela seja segura.

Mais amostras melhoram a estimativa correta, mas não corrigem essa falha. Na versão com corrida, a quantidade de incrementos perdidos pode variar entre as execuções e não segue uma regra determinística.

3. Correção com `critical`

A correção direta coloca cada `dentro++` em uma região `critical`. Apenas uma thread por vez pode executar esse bloco, como se ele estivesse protegido por uma trava. A contagem fica correta, porém milhões de entradas na região crítica serializam grande parte do programa.

A versão final separa `parallel` de `for`. Isso permite criar um contador por thread antes do laço e somá-lo ao total depois:

```
#pragma omp parallel default(none) \
    shared(amostras, dentro, final_x, final_y) \
    firstprivate(semente) private(i, x, y, dentro_local)
{
    dentro_local = 0;

    #pragma omp for lastprivate(final_x, final_y)
    for (i = 0; i < amostras; i++) {
        gerar_ponto(semente, i, &x, &y);
        final_x = x; final_y = y;
        dentro_local += (x * x + y * y <= 1.0);
    }

    #pragma omp critical
    {
        dentro += dentro_local;
    }
}
```

Cada thread altera apenas seu `dentro_local` e entra no `critical` uma única vez. Com 20 threads, são 20 atualizações protegidas, em vez de aproximadamente 7,85 milhões.

Sem a exigência de usar `critical`, uma soma como esta seria normalmente escrita com `reduction(+:dentro).atomic` também poderia proteger cada incremento.

4. Explicação rápida dos conceitos

Condição de corrida: acontece quando duas ou mais threads acessam o mesmo dado ao mesmo tempo, pelo menos uma delas escreve e não existe sincronização. Neste programa, a corrida está em `dentro++`: threads podem ler o mesmo valor e um dos incrementos acaba perdido.

critical: faz somente uma thread por vez executar o bloco protegido. Ele corrige a atualização de `dentro`, mas pode deixar o programa lento quando usado dentro de todas as iterações. Na versão final, cada thread entra nele apenas uma vez para somar seu resultado local.

shared: todas as threads acessam a mesma instância da variável. `amostras` pode ser compartilhada com segurança porque é apenas lida. `dentro` também é compartilhada, mas suas escritas precisam do `critical`.

private: cria uma cópia independente para cada thread, sem copiar o valor que existia antes. Por isso cada thread possui seu próprio `dentro_local` e deve inicializá-lo com zero. `i`, `x` e `y` também são privados para que uma thread não altere os valores usados por outra.

firstprivate: funciona como `private`, mas copia o valor inicial para cada thread. Assim, todas recebem uma cópia da semente 2026. Se fosse usado somente `private(semente)`, seria necessário inicializá-la dentro da região antes do uso.

lastprivate: ao terminar o `for`, copia para a variável externa o valor da última iteração na ordem normal do laço. Aqui, `final_x` e `final_y` recebem as coordenadas da amostra `amostras-1`. Não significa a thread que terminou por último, e essas coordenadas não participam da estimativa de π ; elas apenas demonstram a cláusula.

default(none): obriga o programador a declarar explicitamente o escopo das variáveis externas usadas na região paralela. Isso ajuda o compilador a encontrar variáveis esquecidas e torna o código mais claro. Ele não corrige corridas automaticamente: uma variável `shared` ainda precisa de sincronização quando há escritas concorrentes.

Em resumo, `shared` mantém um dado comum; `private` cria cópias sem valor inicial; `firstprivate` cria cópias com valor inicial; `lastprivate` copia para fora o resultado da última iteração; e `critical` protege uma operação compartilhada.

5. Testes e resultados

O programa foi compilado com:

```
gcc -O2 -Wall -Wextra -std=c99 -fopenmp pi_monte_carlo.c -o pi_monte_carlo.exe
```

Foram realizadas cinco execuções com 10.000.000 de amostras, semente 2026 e 20 threads no Intel Core i7-13650HX. O tempo de parede foi medido por `omp_get_wtime()`.

Combinação testada	Resultado	Mediana do tempo
Sequencial	$\pi = 3,141793600$	0,026 s
<code>shared</code> sem proteção	π entre 0,237711600 e 0,419337600	0,049 s
<code>shared</code> + <code>critical</code> por ponto	$\pi = 3,141793600$	0,750 s
<code>private</code> local + <code>critical</code> por thread	$\pi = 3,141793600$	0,004 s

A referência contou 7.854.484 pontos dentro do círculo em todas as repetições. A versão com corrida contou entre 594.279 e 1.048.344 e variou a cada execução. As duas versões protegidas reproduziram exatamente a referência.

O `critical` por ponto foi correto, mas muito mais lento por causa da serialização. A agregação privada reduziu a sincronização e obteve cerca de 6,5 vezes o desempenho sequencial. Como esses tempos são curtos, o valor exato do speedup pode variar com a carga da máquina.

O `lastprivate` devolveu o ponto `(0,208288741; -0,438420464)`, igual ao ponto calculado separadamente para o índice 9.999.999. A semente externa permaneceu 2026, como esperado com `firstprivate`.

6. Conclusão

As amostras de Monte Carlo são independentes, mas o contador usado para reuni-las não é. A paralelização direta criou uma corrida e produziu valores incorretos. `critical` restaurou a correção, e o uso de um contador privado por thread evitou que a trava fosse acessada a cada iteração. As cláusulas de escopo deixaram explícito quais dados pertencem a cada thread e quais são compartilhados; `default(none)` reforçou essa documentação com verificação do compilador.

7. Código-fonte

pi_monte_carlo.c · 201 linhas

```
1  /*
2  * Tarefa 6 - Estimativa estocastica de pi com OpenMP
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra -std=c99 -fopenmp pi_monte_carlo.c -o pi_monte_carlo.exe
6  *
7  * Executar:
8  * ./pi_monte_carlo.exe [amostras] [threads] [semente]
9  */
10
11 #include <omp.h>
12 #include <stdint.h>
13 #include <stdio.h>
14 #include <stdlib.h>
15
16 /* Gera numeros pseudoaleatorios sem usar um estado global compartilhado. */
17 static uint64_t misturar(uint64_t valor)
18 {
19     valor = (valor ^ (valor >> 30)) * UINT64_C(0xbf58476d1ce4e5b9);
20     valor = (valor ^ (valor >> 27)) * UINT64_C(0x94d049bb133111eb);
21     return valor ^ (valor >> 31);
22 }
23
24 static double uniforme(uint64_t valor)
25 {
26     return (double)(misturar(valor) >> 11) / 9007199254740992.0;
27 }
28
29 /* O ponto depende apenas da semente e do índice da amostra. */
30 static void gerar_ponto(uint64_t semente, long long i, double *x, double *y)
31 {
32     uint64_t base = semente + 2 * (uint64_t)i;
33     *x = 2.0 * uniforme(base) - 1.0;
34     *y = 2.0 * uniforme(base + 1) - 1.0;
35 }
36
37 static long long contar_sequencial(long long amostras, uint64_t semente)
38 {
39     long long dentro = 0;
40
41     for (long long i = 0; i < amostras; i++) {
42         double x;
43         double y;
44         gerar_ponto(semente, i, &x, &y);
45         if (x * x + y * y <= 1.0) {
46             dentro++;
47         }
48     }
49     return dentro;
50 }
51
52 /* Incorreta: todas as threads alteram o mesmo contador sem protecao. */
53 static long long contar_com_corrida(long long amostras, uint64_t semente)
54 {
55     long long dentro = 0;
56
57     #pragma omp parallel for
58     for (long long i = 0; i < amostras; i++) {
59         double x;
60         double y;
61         gerar_ponto(semente, i, &x, &y);
62         if (x * x + y * y <= 1.0) {
63             dentro++;
64         }
65     }
66     return dentro;
67 }
68
69 /* Correta, mas cara: usa a regiao critica em cada ponto aceito. */
70 static long long contar_critical_por_ponto(long long amostras,
71                                           uint64_t semente)
72 {
73     long long dentro = 0;
74
75     #pragma omp parallel for default(none) \
76         shared(amostras, dentro) firstprivate(semente)
77     for (long long i = 0; i < amostras; i++) {
78         double x;
79         double y;
80         gerar_ponto(semente, i, &x, &y);
81         if (x * x + y * y <= 1.0) {
82             #pragma omp critical
```

```

83     {
84         dentro++;
85     }
86 }
87 }
88 return dentro;
89 }
90
91 /* Correta e eficiente: cada thread acumula antes de usar critical. */
92 static long long contar_critical_local(long long amostras, uint64_t semente,
93                                       double *ultimo_x, double *ultimo_y)
94 {
95     long long dentro = 0;
96     long long dentro_local;
97     long long i;
98     double x;
99     double y;
100     double final_x = 0.0;
101     double final_y = 0.0;
102
103     #pragma omp parallel default(none) \
104         shared(amostras, dentro, final_x, final_y) \
105         firstprivate(semente) private(i, x, y, dentro_local)
106     {
107         dentro_local = 0;
108
109         #pragma omp for lastprivate(final_x, final_y)
110         for (i = 0; i < amostras; i++) {
111             gerar_ponto(semente, i, &x, &y);
112             final_x = x;
113             final_y = y;
114             if (x * x + y * y <= 1.0) {
115                 dentro_local++;
116             }
117         }
118
119         #pragma omp critical
120         {
121             dentro += dentro_local;
122         }
123     }
124
125     *ultimo_x = final_x;
126     *ultimo_y = final_y;
127     return dentro;
128 }
129
130 static void imprimir(const char *nome, long long dentro, long long amostras,
131                    double tempo, long long referencia)
132 {
133     double pi = 4.0 * (double)dentro / (double)amostras;
134     printf("%-25s %12lld %.9f %10.6f %lld\n",
135           nome, dentro, pi, tempo, dentro - referencia);
136 }
137
138 int main(int argc, char **argv)
139 {
140     long long amostras = argc > 1 ? atoll(argv[1]) : 10000000;
141     int threads = argc > 2 ? atoi(argv[2]) : omp_get_num_procs();
142     uint64_t semente = argc > 3 ? strtoull(argv[3], NULL, 10) : 2026;
143     long long sequencial;
144     long long corrida;
145     long long critico_por_ponto;
146     long long critico_local;
147     double tempo_sequencial;
148     double tempo_corrida;
149     double tempo_critico_por_ponto;
150     double tempo_critico_local;
151     double ultimo_x;
152     double ultimo_y;
153     double esperado_x;
154     double esperado_y;
155     double inicio;
156
157     if (argc > 4 || amostras <= 0 || threads <= 0) {
158         fprintf(stderr, "Uso: %s [amostras] [threads] [semente]\n", argv[0]);
159         return EXIT_FAILURE;
160     }
161
162     omp_set_dynamic(0);
163     omp_set_num_threads(threads);
164
165     inicio = omp_get_wtime();
166     sequencial = contar_sequencial(amostras, semente);
167     tempo_sequencial = omp_get_wtime() - inicio;
168
169     inicio = omp_get_wtime();
170     corrida = contar_com_corrida(amostras, semente);
171     tempo_corrida = omp_get_wtime() - inicio;

```

```

172     inicio = omp_get_wtime();
173     critico_por_ponto = contar_critical_por_ponto(amostras, semente);
174     tempo_critico_por_ponto = omp_get_wtime() - inicio;
175
176     inicio = omp_get_wtime();
177     critico_local = contar_critical_local(amostras, semente,
178                                         &ultimo_x, &ultimo_y);
179     tempo_critico_local = omp_get_wtime() - inicio;
180
181     gerar_ponto(semente, amostras - 1, &esperado_x, &esperado_y);
182
183     printf("Estimativa estocastica de pi\n");
184     printf("Amostras: %lld | Threads: %d | Semente: %llu\n",
185           amostras, threads, (unsigned long long)semente);
186     printf("Versao          Dentro          pi      Tempo (s)      Diferenca\n");
187     imprimir("Sequencial", sequencial, amostras, tempo_sequencial, sequencial);
188     imprimir("parallel for (corrida)", corrida, amostras, tempo_corrida,
189           sequencial);
190     imprimir("critical por ponto", critico_por_ponto, amostras,
191           tempo_critico_por_ponto, sequencial);
192     imprimir("private + critical", critico_local, amostras,
193           tempo_critico_local, sequencial);
194
195     printf("\nUltimo ponto via lastprivate: (%.9f, %.9f)\n", ultimo_x, ultimo_y);
196     printf("Ultimo ponto de referencia:   (%.9f, %.9f)\n", esperado_x, esperado_y);
197     printf("Semente original: %llu\n", (unsigned long long)semente);
198
199     return EXIT_SUCCESS;
200 }
201 }

```